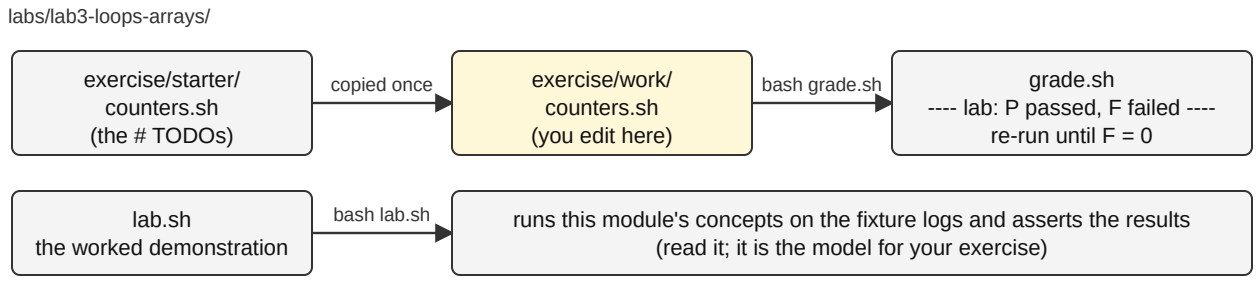
Lab 3 — Loops and Associative Arrays: The Counting Core

Topology



Environment Table

All paths are relative to `labs/lab3-loops-arrays/exercise/`.

File	What it is	You edit it?
`TASK.md`	The short statement of the exercise	No
`starter/counters.sh`	`count_status` and `bruteforce_count`, each with `# TODO`s	No — the grader copies it
`work/counters.sh`	**Your** copy (created by the first `grade.sh` run)	**Yes**
`grade.sh`	The grader: 5 checks. It generates the fixture logs with `seed/make_logs.sh` on every run	No
`work/REFLECTION.md`	Your two reflection answers	**Yes** (you create it)

Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `count_status` — tally HTTP status codes with a `while read` loop and an associative array.
- **Part 3:** Build `bruteforce_count` — count failed sshd logins from one IP.
- **Part 4:** Verify with the grader; write the two reflection answers.

Background / Scenario

The heart of a log analyser is a loop that reads a file one line at a time and counts things. `logsleuth`'s `summary` report counts status codes; its `bruteforce` report counts failed logins per source IP in an sshd log. Both are the same move: `while IFS= read -r line` to read lines safely, and an **associative array** (`local -A n`) to keep a count per key.

The grader generates two fixture logs on every run, always with the same numbers:

- `access.log` — 140 requests to the fraud-clf API gateway; status codes **200 × 85**, **401 × 40**, **500 × 5**. Field 9 of each line is the status code.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

- `auth.log` — an sshd log in which IP **203.0.113.9** has **25** `Failed password` lines and `10.0.0.1` has none.

Required Resources

- A lab session or a shell where `bash tools-check.sh` reports `ready`.
- Module M03, steps S03.1–S03.4.
- A text editor. Documentation and AI assistance for lookups are allowed (say so in your reflection). Do **not** open `solution/`.

Part 1: Prepare the exercise folder

Step 1: Run the grader once.

```
cd labs/lab3-loops-arrays/exercise
bash grade.sh
```

Expected: `---- lab3: 1 passed, 4 failed ----`.

Step 2: Open `work/counters.sh`.

`count_status` already has the loop skeleton (`while IFS= read -r line; do … done < "$f"`) with two `# TODO`s inside; `bruteforce_count` prints `0` and has one `# TODO`.

Part 2: Build `count_status`

`count_status FILE` prints one `<code> <count>` line per status code, sorted by code.

Step 1: Split each line and count field 9.

Replace the first `:` placeholder inside the loop with:

```
read -ra parts <<< "$line"
code="${parts[8]:-}"
[[ "$code" =~ ^[0-9]{3}$ ]] || continue
n["$code"]=$(( ${n["$code"]:-0} + 1 ))
```

How it works: `read -ra parts <<< "$line"` splits the line on whitespace into the indexed array `parts`; field 9 is index 8 (arrays start at 0). The `=~` guard skips any line whose field 9 is not three digits, so a malformed line cannot become a bogus key. `${n["$code"]:-0}` reads the current count, or `0` if that key has never been seen — without the `:-0`, the first increment of a new key is an **unbound variable** under `set -u` and the script stops.

Step 2: Print the tally, sorted by code.

Replace the second `:` (after `done < "$f"`) with:

```
for c in $(printf '%s\n' "${!n[@]}" | sort); do
  printf '%s %s\n' "$c" "${n[$c]}"
done
```

`"${!n[@]}"` lists the **keys** of the array; `sort` orders them; the loop prints each key with its count.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Note: Declare the arrays at the top of the function with `local -A n` (already in the starter) and `local -a parts`; `-A` is what makes `n` associative.

Step 3: Try it by hand.

```
tmp=$(mktemp -d) && bash ../../../seed/make_logs.sh "$tmp" >/dev/null
source work/counters.sh
count_status "$tmp/access.log"
```

Expected: three lines, `200 85`, `401 40`, `500 5`.

Part 3: Build `bruteforce_count`

`bruteforce_count FILE IP` prints how many `Failed password … from IP …` lines the auth log has for that IP.

Step 1: Loop and match.

Replace the body (keep the `local` line) with:

```
bruteforce_count() {
  local f="$1" ip="$2" line count=0
  while IFS= read -r line; do
    [[ "$line" == *"Failed password"*"from $ip "* ]] && count=$(( count + 1 ))
  done < "$f"
  printf '%s\n' "$count"
}
```

The pattern `*"Failed password"*"from $ip "*` matches a line that contains both fragments in that order. The **space after `$ip`** matters: `from 203.0.113.9` must not also match `203.0.113.99`.

Step 2: Try it.

```
bruteforce_count "$tmp/auth.log" 203.0.113.9
bruteforce_count "$tmp/auth.log" 10.0.0.1
rm -rf "$tmp"
```

Expected: `25`, then `0`.

Part 4: Verify and reflect

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab3: 5 passed, 0 failed ----`. The checks: `200 85`, `401 40` and `500 5` present in `count_status` output; `203.0.113.9` has 25 failed logins; an innocent IP has 0.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Symptom	Cause	Fix
`200 count is 85` got `[]`	The loop counts but the print loop is still `:`	Add the `for c in …` loop after `done`
`n: unbound variable`	A new key read without a default	`${n["$code"]:-0}`
Counts are off by a few	Lines with a non-numeric field 9 were counted	Keep the `=~ ^[0-9]{3}$` guard
`25` expected, more returned	The pattern has no space after `$ip`	`"from $ip "`

Step 2: Reflection.

Create `work/REFLECTION.md` and answer in your own words (3–5 sentences each). Question 2 asks for an actual observation — run the experiment.

Question 1: Why `while IFS= read -r line … done < "$f"` rather than `for line in $(cat "$f")`? Name two things the `for` version breaks.

Question 2: Why does the counter use `${n["$code"]:-0}`? Test what happens under `set -u` without it, and describe what you saw.

Hand-in and grading

Hand in `work/counters.sh`, `work/REFLECTION.md` and your final grader output. **Pass** = `0 failed` **and** both reflection answers name the mechanism (Question 2 with an observation). Unlimited attempts.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.